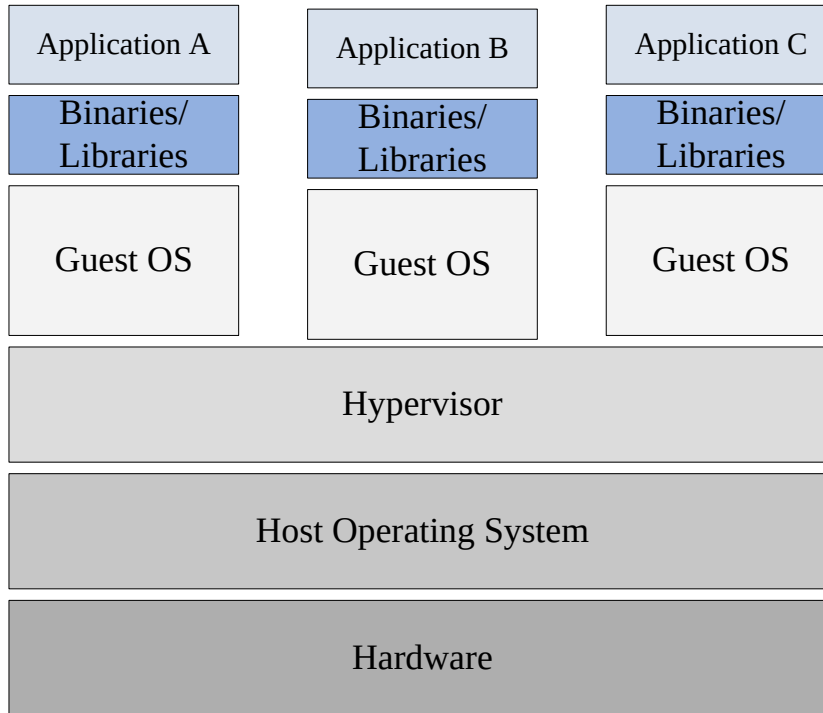
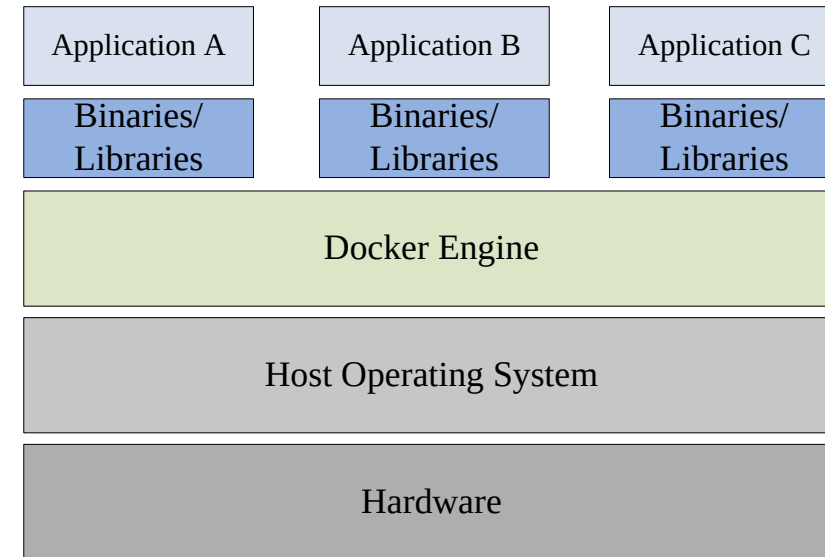


Week #3

Virtual Machines and Docker containers.



(left) VM;



(right) Multiple Docker containers running on the same machine share the OS kernel thus, have a smaller memory footprint and a shorter start-up time than VMs

Containers

- Containers
 - Are based on OS-level virtualization rather than hardware virtualization
 - Isolate applications running inside a container from applications running in a different container
 - Isolate applications from the physical system where they run
 - Resources used by a container can be limited
- Benefits
 - Ease creation and deployment of applications.
 - Applications container images are created at build time, rather than deployment time.
 - Support portability; containers run independently of the environment.
 - Benefit from an application-centric management.
 - Optimal philosophy for application deployment; applications are broken into smaller, independent pieces and can be managed dynamically.
 - Support higher resource utilization.
 - Lead to predictable application performance.

Containers

- Containers are based on operating-system-level virtualization, while virtual machines are based on hardware virtualization.
- Isolate an application running inside a container from another application running and isolate from the physical system.
- Support higher resource utilization
- Lead to predictable application performance.
- Are portable and the resources used by a container can be limited.
- Are more transparent than VMs thus, easier to monitor and manage.
- Ease the creation and the deployment of applications
- Application container images are created at build time rather than deployment time.
- Benefit from an application-centric management.
- Have an optimal philosophy for application deployment; applications are broken into smaller, independent pieces and can be managed dynamically.

Docker containers

- The *cgroups (control groups)* concept was implemented in the Linux kernel in 2006 at Google to isolate, control, limit, prioritize, and account for the resources (CPU, memory, disk I/O, network, etc.) available to a set of processes.
 - Control allows freezing groups of processes, manages checkpointing, and restarting.
 - Resource limiting enforces the target set for resource utilization, while prioritization allows a group of processes to get a larger share of CPU cycles or a higher disk I/O throughput.
- Containerization
 - Allows developers to choose the most suitable programming languages and software systems.
 - Eliminates the need to make copies of production code and install the same configuration in different environments.
 - Supports efficient up and down scaling of applications.
- Multiple Docker containers running on the same machine share the OS kernel thus, have a smaller memory footprint and a shorter start-up time than VMs

Container services

- ECS is Amazon EC2 Container Service.
 - ECS is integrated with existing services such as IAM for permissions, CloudTrail to get data regarding resources used by a container, CloudFormation for cluster launching, and other services. AWS uses a custom scheduler/cluster manager for containers.
 - Container hosts are regular EC2 instances.
 - To deploy a containerized application on AWS one has to first publish the image on an AWS accessible registry e.g., the Docker Hub.
- Google Container Engine is based on Kubernetes an open source cluster manager available to Google developers and the community of outside users. Google's approach to containerization emphasis is more on performance than ease of use. Two billion containers are started at Google every week.
- Microsoft Azur Container Service. The Azure Resources Manager API supports multiple orchestrations including Docker, and Apache Mesos.

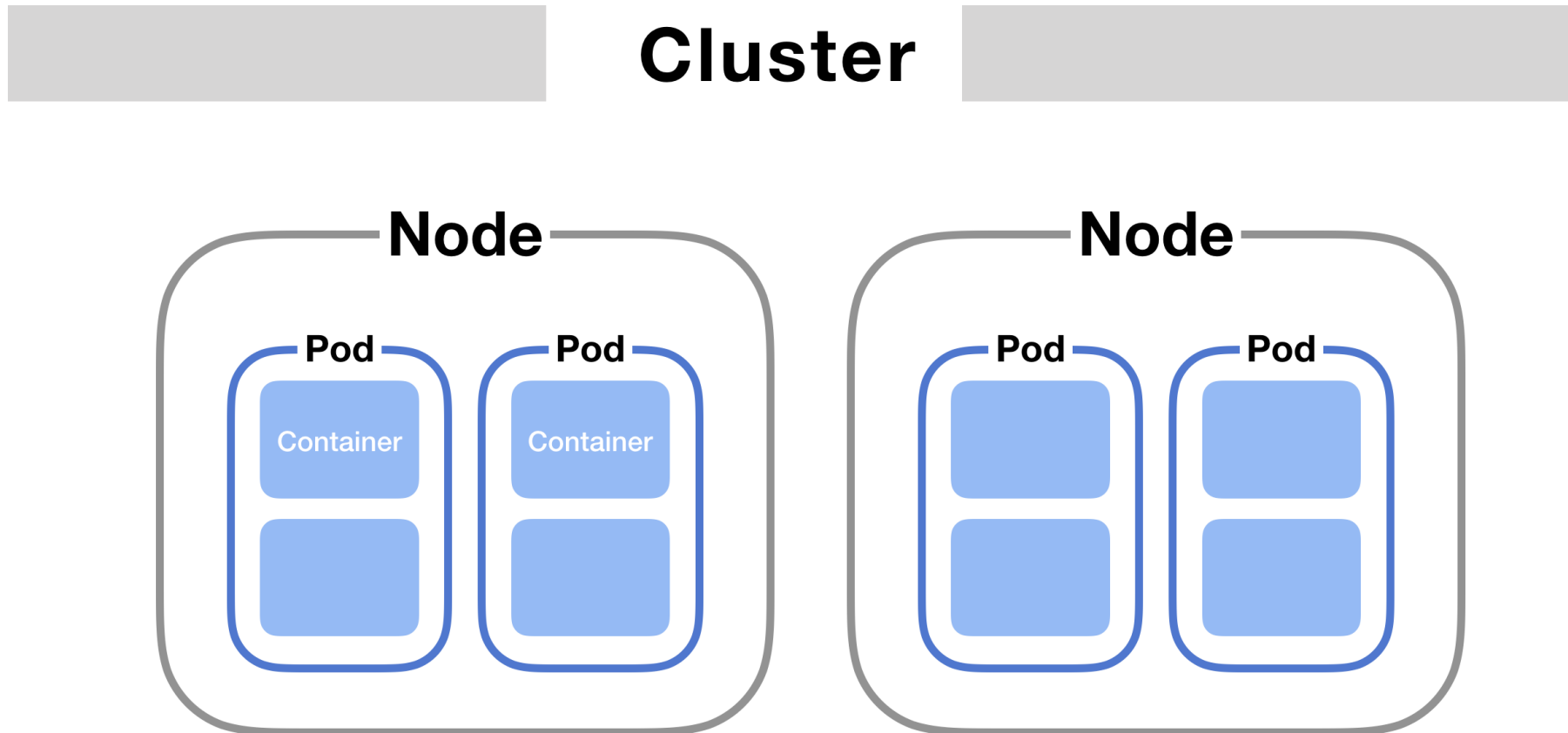
Kubernetes

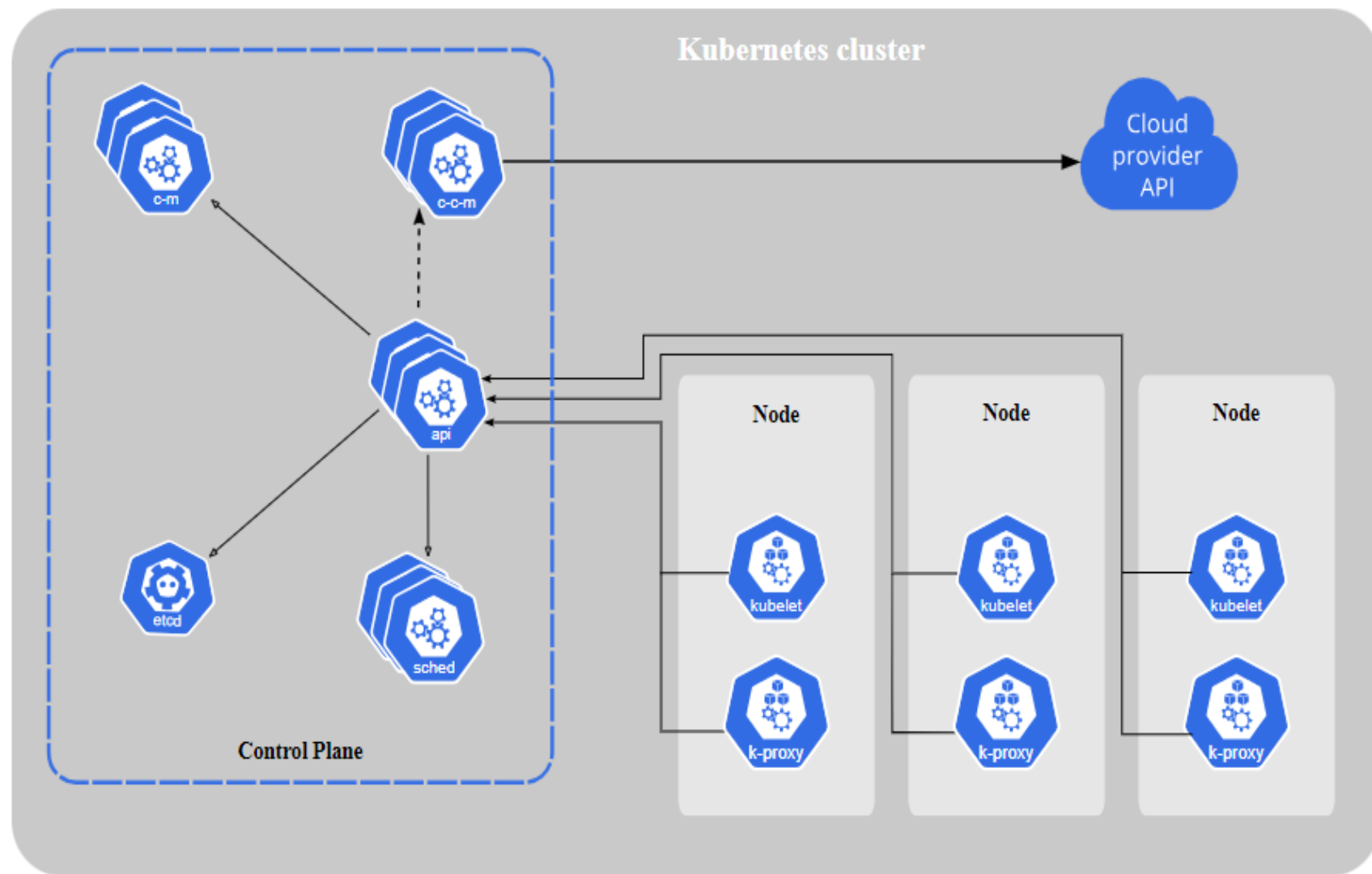
- Open-ended system; its design allowed a number of other systems to be build atop Kubernetes.
- A master server manages a Kubernetes cluster and provides services to manage the workload and to support communication for a large number of relatively unsophisticated minions servers that do the actual work.
- Etcd is a lightweight, distributed key-value store to share configuration data with the cluster nodes.
- The master also provides an API service; an HTTP/JSON API is used for service discovery.
- The scheduler tracks resources available and those allocated to the workloads on each host.

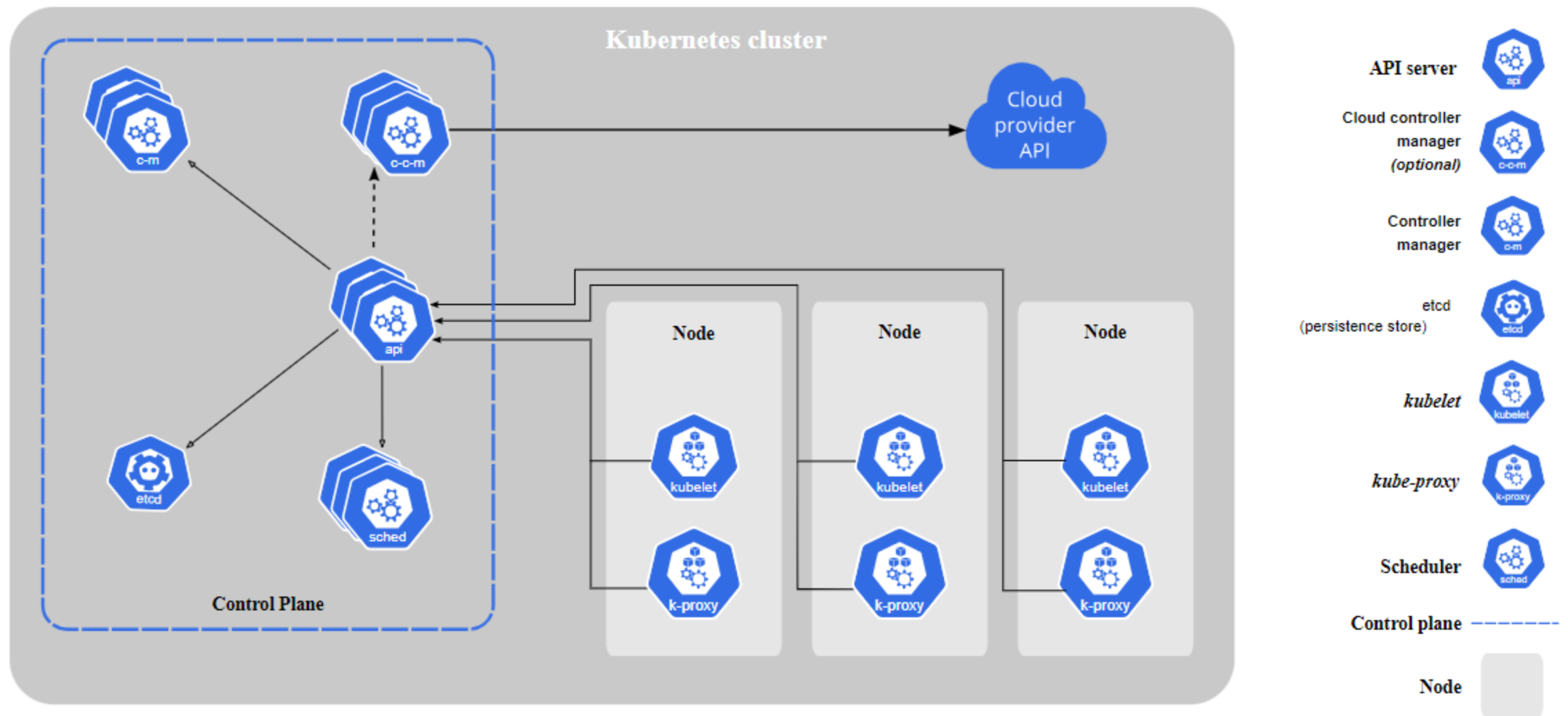
Kubernetes pods

- Pods are groups of containers scheduled onto the same host and serve as units of scheduling, deployment, horizontal scaling, and replication. Pods share fate and resources such as storage volumes.
- The *run* command is used to create a single container pod and the Deployment which monitors that pod. All applications in a pod share a network namespace, including the IP address and the port space thus, can communicate with each other using *localhost*.
- Pods manage co-located support software including: content management systems, controllers, managers, configurators, updaters, logging and monitoring adapters, and event publishers. Pods also manage file and data loaders, local cache managers, log and checkpoint backup, compression, rotation, snapshotting, data change watchers, log trailers, proxies, bridges, and adapters
- Pods are preferable to running multiple applications in a single Docker container.

Kubernetes Containers, Pods and Cluster







Docker containers

- LIGHTWEIGHT

- Containers running on a single machine share the same operating system kernel; they start instantly and use less RAM.
- Images are constructed from layered file systems and share common files, making disk usage and image downloads much more efficient.

- OPEN

- Docker containers are based on open standards, enabling containers to run on all major Linux distributions and on Microsoft Windows -- and on top of any infrastructure.

- SECURE BY DEFAULT

- Containers isolate applications from one another and the underlying infrastructure, while providing an added layer of protection for the application.

Docker containers

- ACCELERATE DEVELOPERS

- Stop wasting hours setting up developer environments, spinning up new instances, and making copies of production code to run locally. With Docker, you simply take copies of your live environment and run them on any new endpoint running a Docker engine.

- EMPOWER CREATIVITY

- The isolation capabilities of Docker containers free developers from constraints: they can use the best language and tools for their application services without worrying about causing internal tooling conflicts.

- DISTRIBUTE & SHARE CONTENT

- Store, distribute, and manage Docker images in Docker Hub with your team. Image updates, changes, and history are automatically shared across your organization.

COMPARING CONTAINERS AND VIRTUAL MACHINES

Containers and virtual machines have similar resource isolation and allocation benefits -- but a different architectural approach allows containers to be more portable and efficient.

VIRTUAL MACHINES

- Virtual machines include the application, the necessary binaries and libraries, and an entire guest operating system -- all of which can amount to tens of GBs.

CONTAINERS

- Containers include the application and all of its dependencies --but share the kernel with other containers, running as isolated processes in user space on the host operating system.
- Docker containers are not tied to any specific infrastructure: they run on any computer, on any infrastructure, and in any cloud.